12 Expert Tips · April 2026

How to use GitHub Copilot better than 99% of people

Agent Mode, Cloud Agent, MCP Servers, Copilot CLI, and the features most developers don't know exist.

by **Shailesh Mishra** | LinkedIn | @sendtoshailesh

⚡

IDE · AGENT MODE

Switch to Agent Mode — stop using plain chat

Agent mode doesn't just suggest code. It edits files, runs terminal commands, installs packages, and self-corrects errors in a loop — all autonomously.

```
// One prompt in Agent mode:
> "Add a /health endpoint with tests"

✓ Created src/routes/health.ts
✓ Created tests/health.test.ts
✓ Installed supertest
✓ Ran npm test — 3/3 passed
✓ Fixed lint error in health.ts → re-ran → clean
```

- Open Copilot Chat → switch dropdown from **Chat** to **Agent**
- Describe *what* you want, not *how* to build it
- Agent creates files, runs tests, and auto-fixes failures
- Works in VS Code, VS Code Insiders, and JetBrains IDEs
- Combine with `#file` and `#codebase` for focused context

💡 PRO TIP

Use Agent mode for multi-file tasks: "Refactor the auth module to use JWT and update all 14 test files"

📖 docs.github.com/en/copilot 🚀 github.com/features/copilot

🤖

CLOUD AGENT

Assign issues directly to Copilot

Copilot Cloud Agent runs in the background on GitHub — no IDE needed. Assign an issue and it researches, codes, and opens a PR with CI passing.

📋 Open Issue → 🤖 Copilot Codes → 🔀 PR + CI → ✅ Review & Merge

- Open any issue → Assignees → select **Copilot**
- Agent signs its commits and runs your CI pipeline
- Works entirely in the cloud — zero IDE time
- Iterates on PR review feedback automatically
- Add `copilot:agent` label to auto-assign future issues

💡 EXAMPLE ISSUE

"Fix the 404 error on /api/users/:id when user has no profile photo. Return default avatar URL."

```
$ gh issue edit 123 --add-assignee @copilot
✓ Copilot assigned to issue #123
✓ PR #124 opened — 3 files changed, CI green
```

📖 Cloud Agent Guide 📖 Creating PRs with Copilot

CONFIG

Add custom instructions — once

One file in your repo tells Copilot your coding standards. Every session reads it automatically — IDE, Cloud Agent, CLI, and Code Review.

```
.github/copilot-instructions.md
.github/instructions/*.instructions.md
AGENTS.md
```

SAMPLE COPILOT-INSTRUCTIONS.MD

```
- Use TypeScript strict mode for all new files
- Follow the repository pattern: controller → service → repo
- Always add JSDoc comments for public methods
- Use Vitest for testing, aim for 80% coverage
- Never use `any` type — use `unknown` instead
```

- Define language, framework, and testing conventions
- Path-specific instructions with glob patterns
- Org-level instructions now GA (April 2026)
- Works across IDE, Cloud Agent, CLI, and Code Review
- Stack multiple instruction files for different areas of your codebase

Custom Instructions Docs

🧠

MODEL SELECTION

Pick the right model for the job

Click the model picker in VS Code or on GitHub. Match the model to the task — each has different strengths and cost multipliers.

🏗️ **COMPLEX / REASONING**

Claude Opus 4.7 — deep refactors, architecture
GPT-5.4 — large multi-file changes
o3 — complex reasoning tasks

⚡ **FAST / EVERYDAY**

GPT-5.4 mini — fast completions
Claude Sonnet 4.5 — balanced
Gemini Flash — high volume, cheap

- Switch models mid-conversation anytime in chat
- Each model has a premium request multiplier (1x–50x)
- CLI supports BYOK & local models via Ollama
- Cloud Agent defaults to the best available model

💡 STRATEGY

`Quick fixes → GPT-5.4 mini (1x) | Architecture → Claude Opus (50x) | Bulk tests → Gemini Flash (0.25x)`

📖 Models & Billing

PROMPT FILES

Create reusable prompt files

Store task-specific prompts as Markdown files. Invoke them with **/** slash command in chat. Shareable across your entire team for consistency.

```
.github/prompts/code-review.prompt.md
.github/prompts/add-tests.prompt.md
.github/prompts/refactor-to-pattern.prompt.md
.github/prompts/deploy-checklist.prompt.md
```

SAMPLE ADD-TESTS.PROMPT.MD

```
Write unit tests for the selected code using Vitest.
- Cover happy path, edge cases, and error scenarios
- Mock external dependencies with vi.mock()
- Use descriptive test names: "should [verb] when [condition]"
- Aim for >90% branch coverage
```

- One file per workflow: review, deploy, migrate, document
- Include variables, constraints, and output format
- Team members use the same prompts for consistent output
- Supports template variables like `{{LANGUAGE}}`
- Works in IDE and with Cloud Agent

Custom Instructions & Prompts

MCP SERVERS

Connect external tools via MCP

Model Context Protocol lets Copilot talk to databases, APIs, Docker, Azure, and 50+ integrations mid-conversation. No more copy-pasting context.

Copilot ↔ MCP ↔ DBs · APIs · Cloud

.VSCODE/MCP.JSON

```
{
  "servers": {
    "github": { "command": "npx", "args": ["github-mcp-server"] },
    "postgres": { "command": "npx", "args": ["@modelcontextprotocol/server-postgres"] }
  }
}
```

- Copilot reads real data from your systems at runtime
- Org admins can manage MCP allow-lists for security
- Works in VS Code, CLI, and Cloud Agent
- GitHub MCP Server: query issues, PRs, repos, and more
- 50+ community servers: Figma, Slack, Jira, Docker, etc.

MCP with Cloud Agent github/github-mcp-server MCP Governance

CLI

Copilot CLI — AI in your terminal

Run `copilot` from any terminal. It reads your repo, makes changes, runs commands, opens PRs — all with your approval before each step.

```
$ copilot
> "Add rate limiting to all API routes and open a PR"

✓ Analyzed 12 route files
✓ Created src/middleware/rate-limit.ts
✓ Updated 12 route files with middleware
✓ Ran npm test — 47/47 passed
✓ Opened PR #42 — "Add rate limiting middleware"
```

- Interactive mode + programmatic mode with `-p "prompt"`
- Plan mode: **Shift+Tab** to preview before execution
- Supports BYOK and local models (Ollama, LM Studio)
- MCP servers work in CLI too — same `.vscode/mcp.json`
- Install: `gh extension install github/gh-copilot`

About Copilot CLI Install Guide

🔍

IDE SHORTCUTS

Master @workspace, @terminal, and slash commands

Give Copilot full repo context with **@workspace**. Use **@terminal** for CLI debug. Fire off instant actions with slash commands.

```
@workspace "how does auth work in this repo?"
@terminal "why did that test fail?"
/fix → auto-fixes selected code
/tests → generates unit tests for selection
/explain → explains code in plain English
/doc → adds JSDoc / docstring
```

- `#file` — attach specific files as context
- `#selection` — reference your current editor selection
- `#codebase` — search the entire project for relevant context
- Combine: `@workspace #file:schema.ts` "add validation for the User model"

💡 POWER COMBO

Select a function → press **Cmd+I** → type "/tests" → instant test file with edge cases

📖 Copilot Docs

📝

CODE REVIEW

Let Copilot review your pull requests

Copilot Code Review catches bugs, security issues, and style violations before human reviewers even look. Enable it org-wide for automatic reviews on every PR.

📤 Push PR → 🤖 Copilot Reviews → 🔧 Suggested Fixes → ✅ Merge

- Catches security vulnerabilities automatically (SQL injection, XSS, etc.)
- Suggests concrete code fixes, not just warnings
- Customize rules with `.github/copilot-instructions.md`
- Enable automatic reviews for all PRs in repo settings
- Review comments are actionable — click to apply fixes

💡 AUTO-REVIEW SETUP

```
Repo Settings → Copilot → Code Review → Enable "Automatic reviews" → Choose triggers (all PRs / labeled / draft)
```

📖 Copilot Code Review Docs

SDK · SKILLS · EXTENSIONS

Build with Agent Skills & Extensions

Copilot is a platform, not just a tool. Create custom agent skills, VS Code chat participants, and internal developer tools that plug into the Copilot ecosystem.

AGENT SKILLS

Custom instructions + scripts
Enhance Cloud Agent abilities
Manage via `gh copilot skill`
Workspace or user-level scoping

COPILOT EXTENSIONS

VS Code extensions with AI
Custom chat participants (@your-tool)
GitHub Apps with Copilot API
Marketplace distribution

SKILL FILE STRUCTURE

```
.copilot/skills/my-skill/SKILL.md    # Skill instructions
.copilot/agents/my-agent.agent.md  # Agent definition
.copilot/instructions.md           # Workspace config
```

- Create specialized agents for your team's workflows
- Skills run locally or in Cloud Agent sandbox
- Chain agents together for multi-step pipelines

Agent Skills Docs

ENTERPRISE

Configure org-level governance

Granular enterprise controls for Cloud Agent, model access, MCP servers, data residency, and audit logging. Power without governance is chaos.

🔒 GOVERNANCE CHECKLIST

```
✅ Set org-level Copilot instructions (enforced for all repos)
✅ Configure Cloud Agent runner settings (self-hosted / GitHub-hosted)
✅ Define MCP server allow-lists per organization
✅ Set firewall rules for agent network access
✅ Enable audit logs for all Copilot activity
✅ Configure data residency (US / EU — FedRAMP compliant)
```

- Org runner controls for Cloud Agent (April 2026)
- Model access policies: allow/deny specific models per org
- Custom properties for fine-grained repo-level Copilot settings
- Audit log integration with SIEM tools
- IP allow-listing for Copilot API access

📖 Agent Settings 📖 MCP Management

BEST PRACTICE

Your repo IS Copilot's brain

Copilot's output quality is directly proportional to your repo quality. Clean structure, good README, consistent patterns, and clear naming — these aren't optional.

❌ MESSY REPO

No README or docs
Inconsistent naming (camelCase + snake_case)
Flat file structure, no modules
No tests, no types
→ Weak, generic suggestions

✅ CLEAN REPO

Detailed README.md with architecture
Consistent naming conventions
Logical folder structure + modules
Full test suite + TypeScript strict
→ Precise, contextual suggestions

🧠 REPO AS CONTEXT: WHAT COPILOT READS

```
README.md → project purpose, architecture
.github/copilot-instructions.md → coding standards
AGENTS.md → agent behavior rules
tsconfig.json / .eslintrc → language conventions
Existing code patterns → naming, structure, style
```

- Write a solid README — it's the first thing Copilot reads
- Use **Copilot Spaces** to share curated context across teams
- Add architecture decision records (ADRs) to guide long-term patterns

📖 Copilot Best Practices

Stop using Copilot like autocomplete.

These 12 features separate the top 1% from everyone else. The gap is only getting wider.

Full Cheatsheet →

♻️ **Repost** if your network needs to see this

💾 **Save** this carousel for later

💬 **Comment** which tip was new to you

CONNECT WITH ME

🔗 LinkedIn 𝕏 Twitter 📷 Instagram

#GitHubCopilot #AI #DeveloperProductivity #CopilotAgent #SoftwareEngineering #DevTools #GitHub